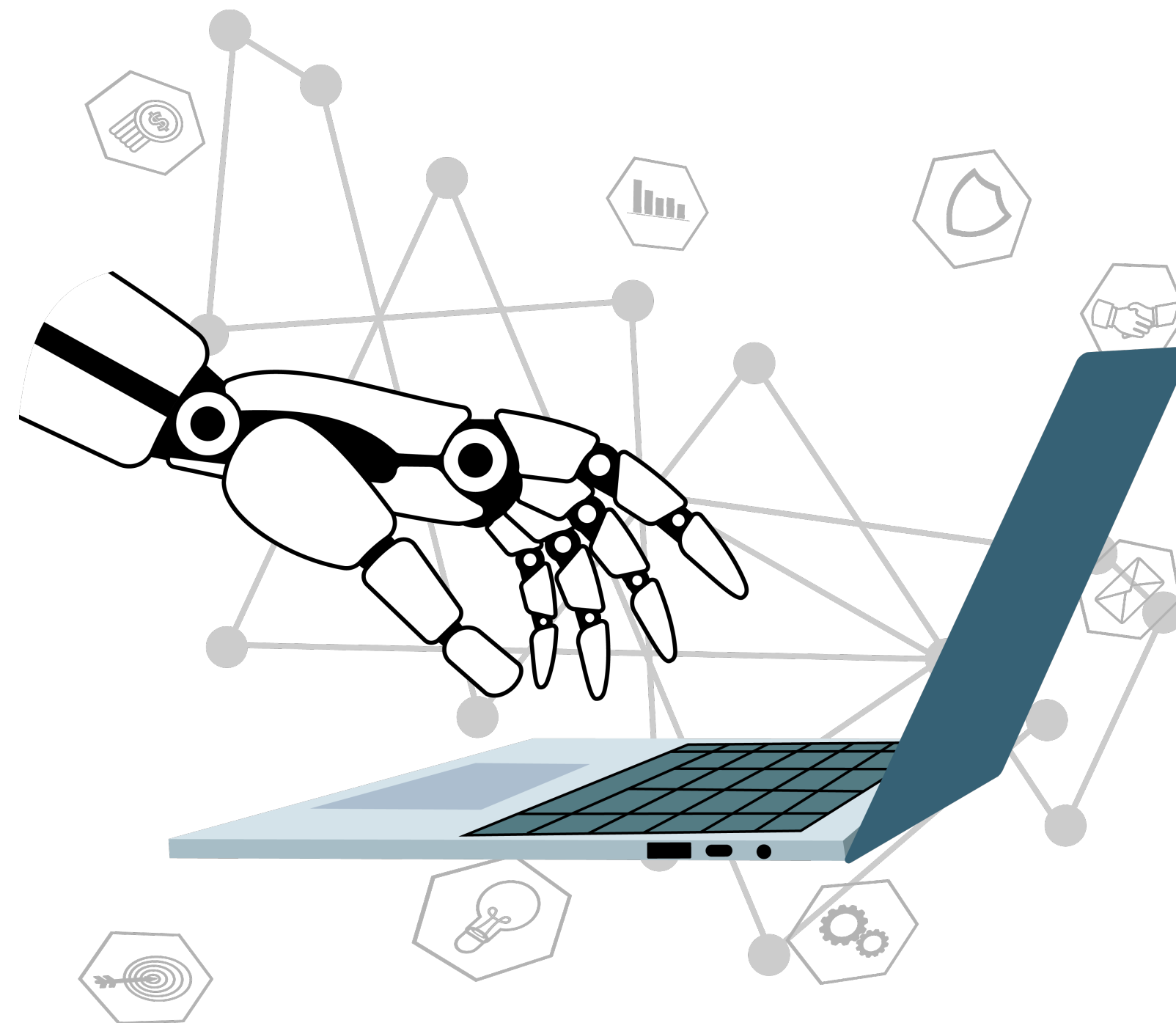


이직 데이터를 활용한 머신러닝 성능 개선

Team. G09

FINAL PROJECT

김현민, 윤성빈, 최찬영



데이터과학개론

⁰¹ Index

Dataset

이직 데이터 소개

프로젝트 목표

Preprocessing

정규화

feature 데이터 추가

이상치 및 결측치

Model

머신러닝 모델

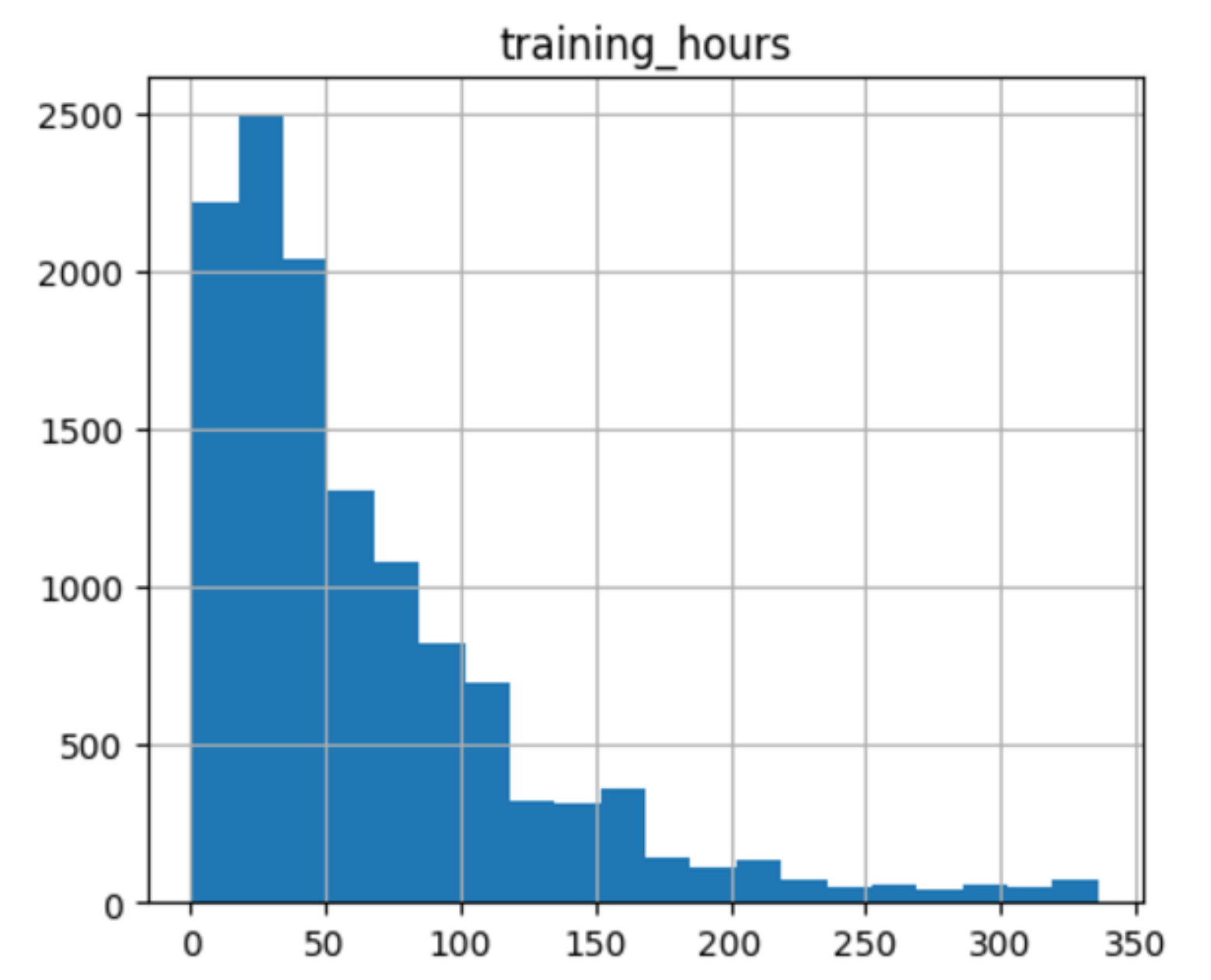
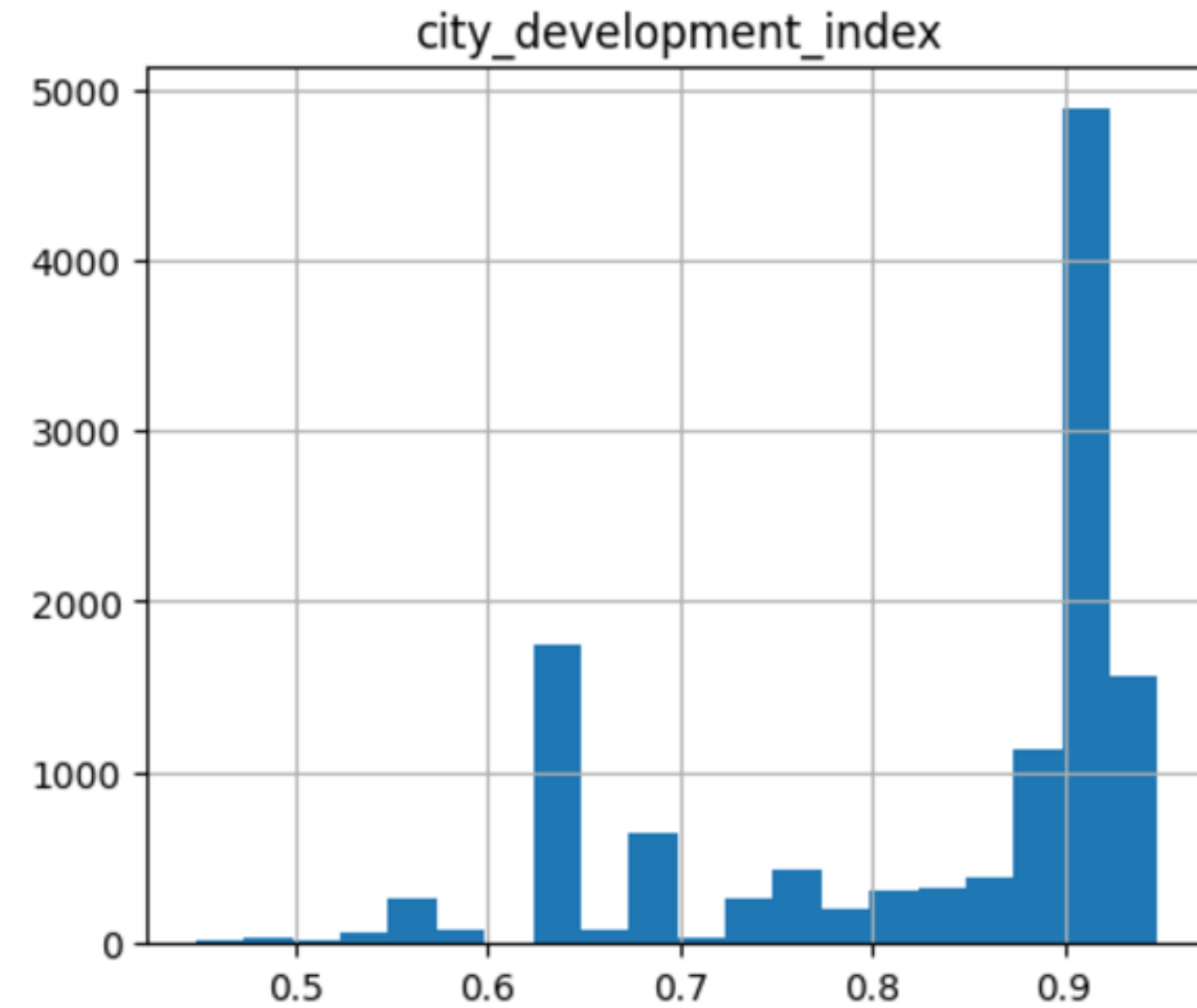
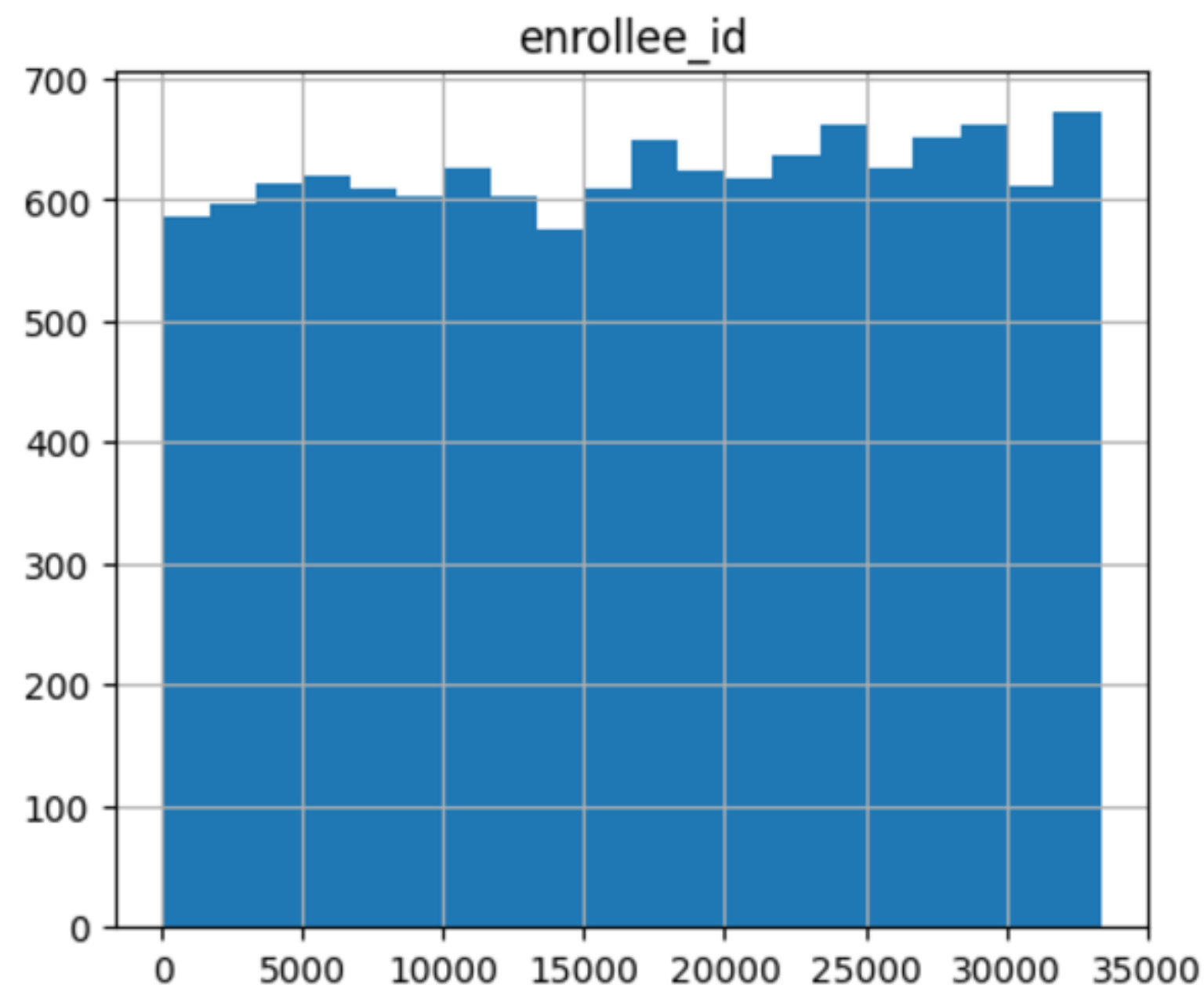
신경망 모델

Optimization

L2 정규화 및 metrics

DATASET

이직 여부 데이터 수치형 데이터에 관한 고찰



CITY_DEVELOPMENT_INDEX

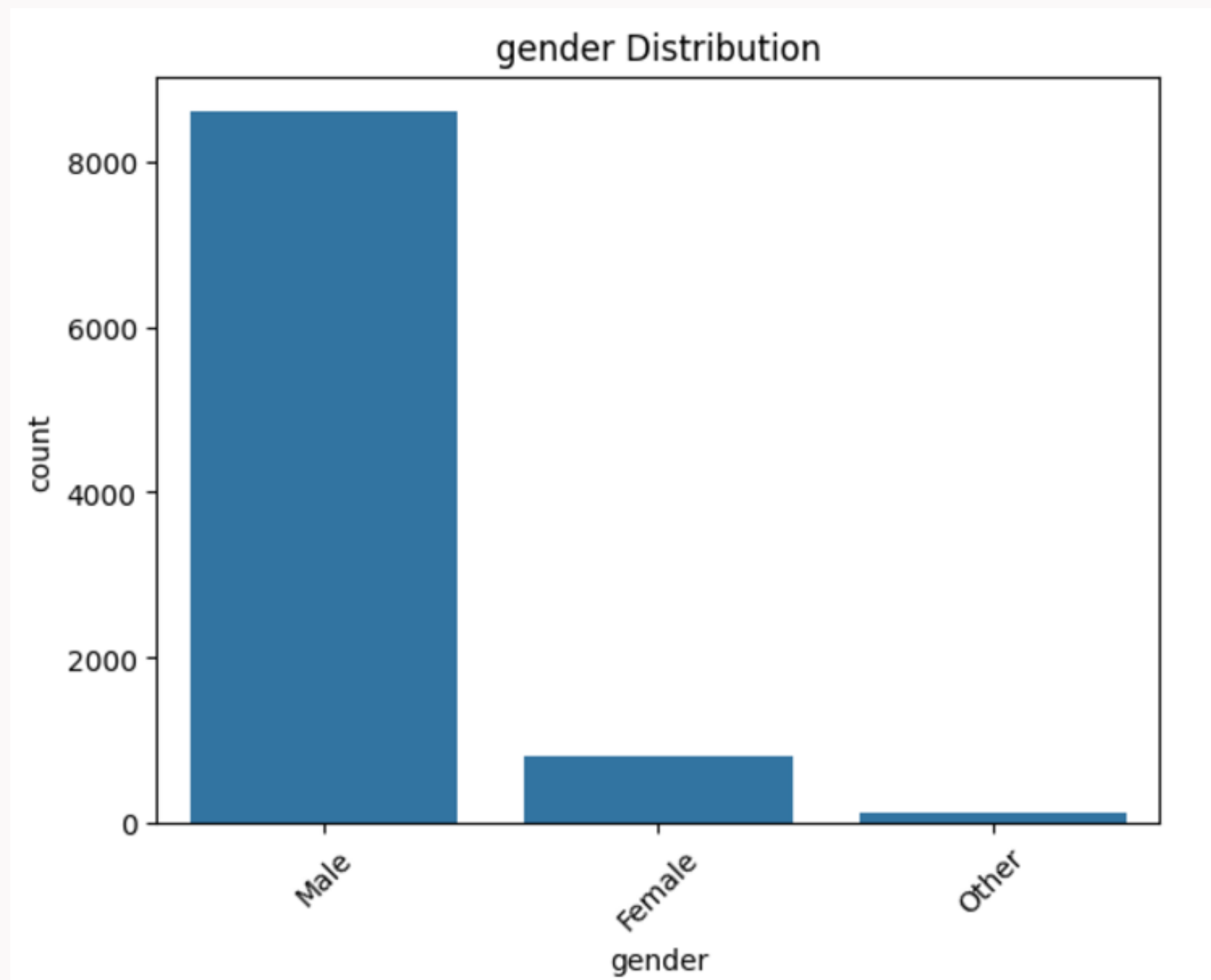
비교적 개발 수준의 도시에 거주하는 사람들의
데이터가 다량으로 분포됨

TRAINING HOURS

적은 교육 시간의 데이터에 몰린 왼쪽으로 치우친
형태의 분포를 보임

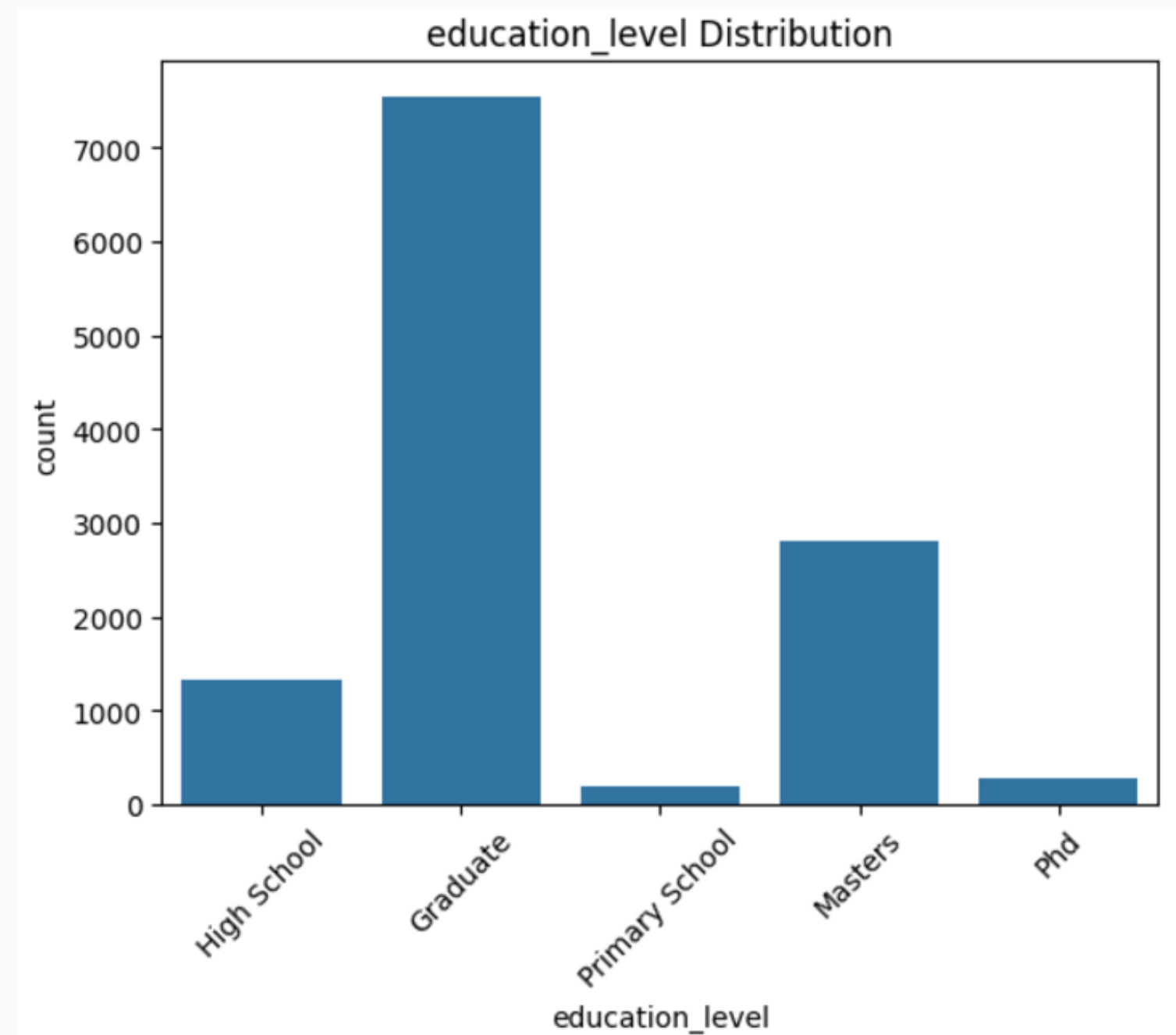
DATASET

이직 여부 데이터 범주형 데이터에 관한 고찰



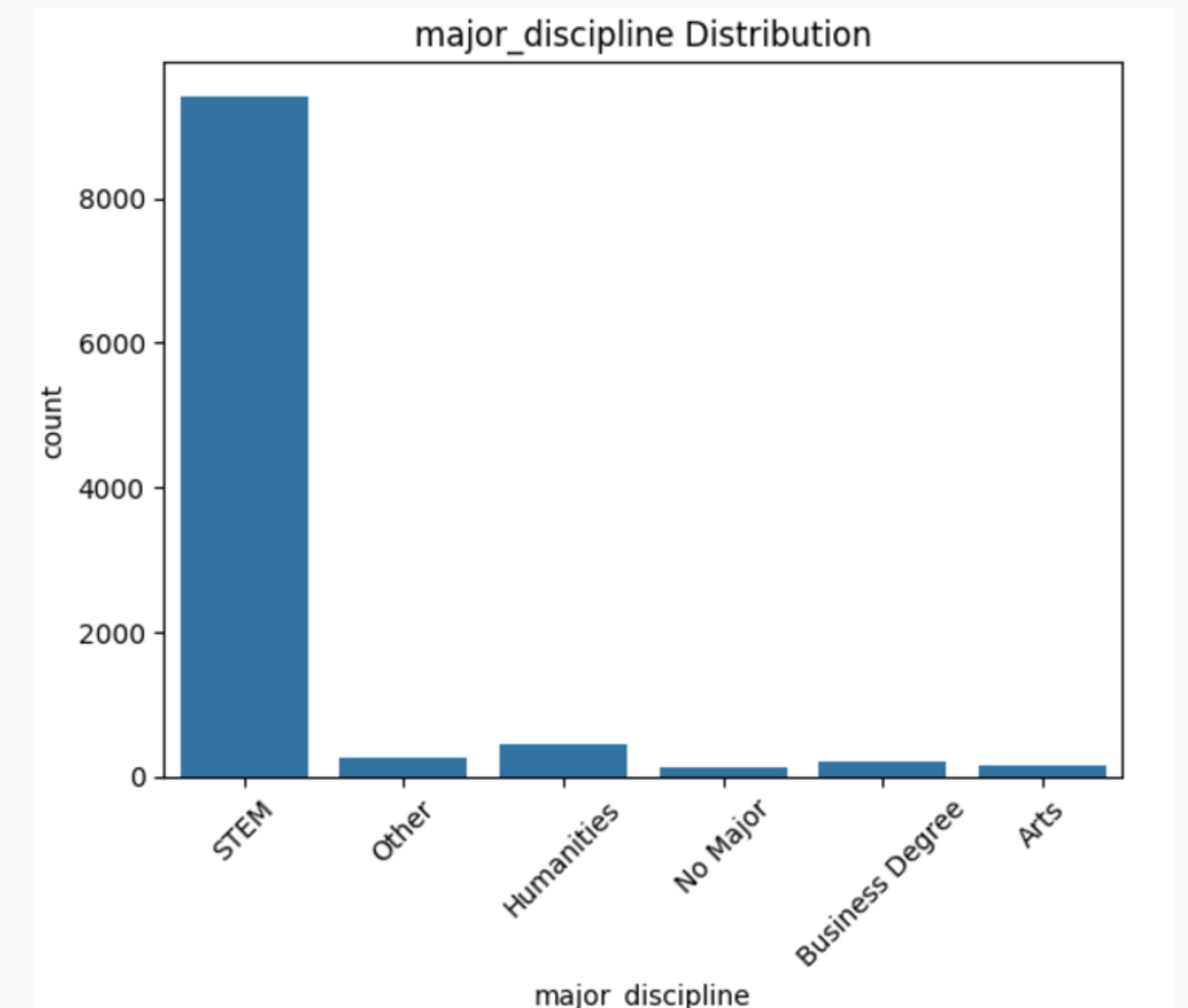
GENDER DISTRIBUTION

주로 여성보다는 남성에 편향적인 데이터
분포를 보임



EDUCATION LEVEL DISTRIBUTION

대학 졸업자의 분포가 가장 많으며, 박사 학위 등
고학위 졸업자들이 그 뒤를 이음

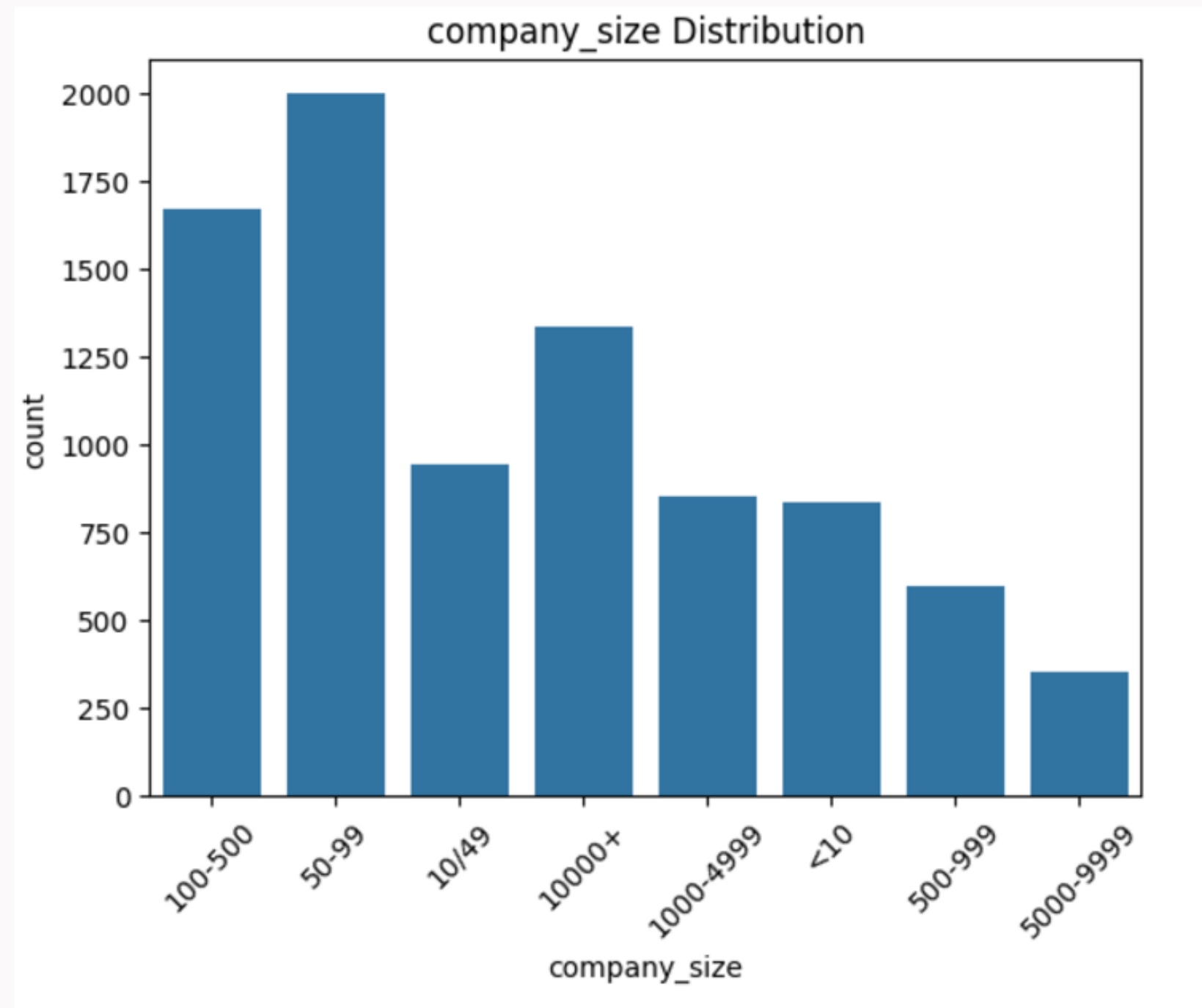


MAJOR DISCIPLINE

타 전공 과목들에 비해 STEM (Science, Technology,
Engineering, Math) 이공계 과목 전공 비중이 높음

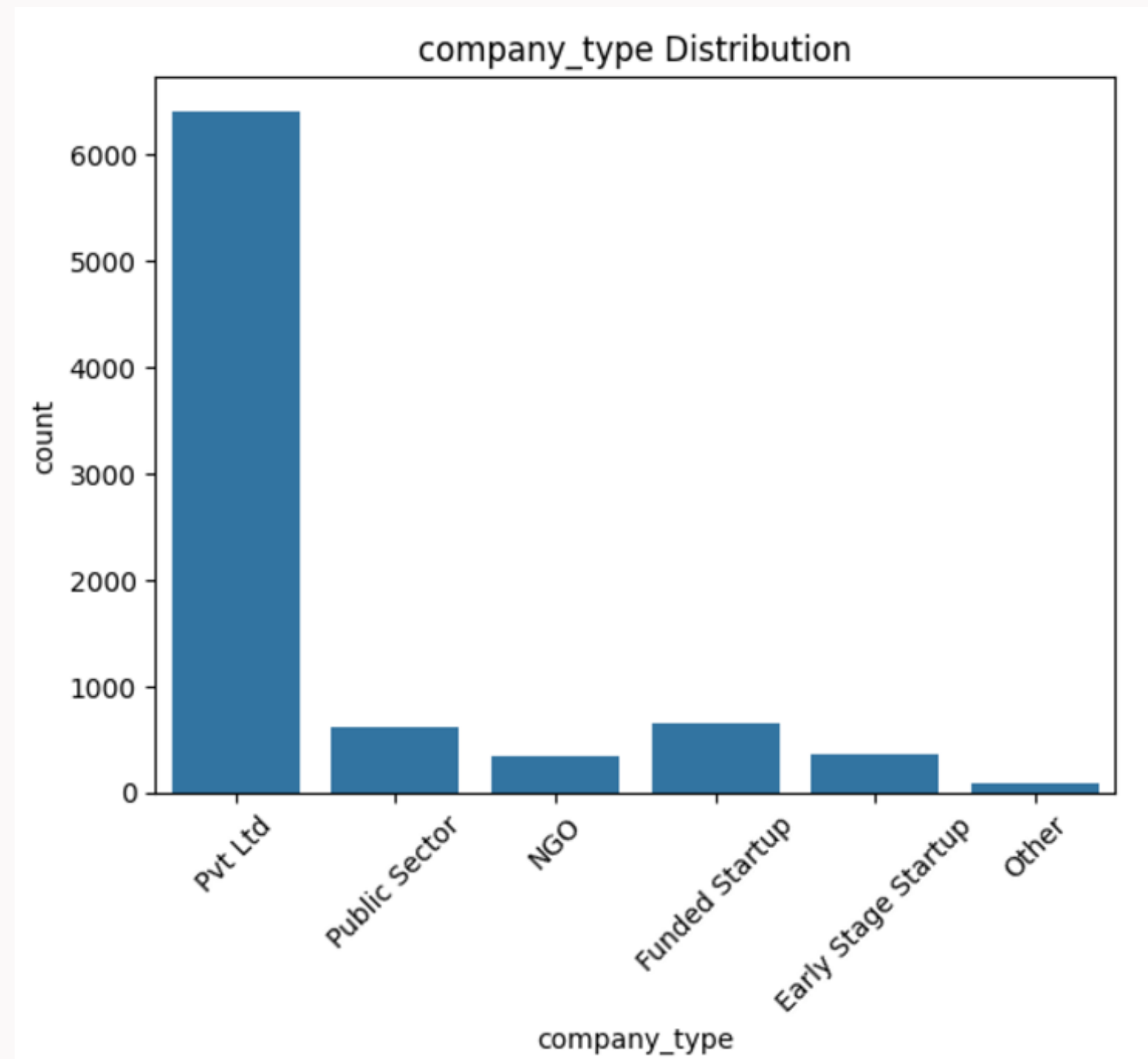
DATASET

이직 여부 데이터 범주형 데이터에 관한 고찰



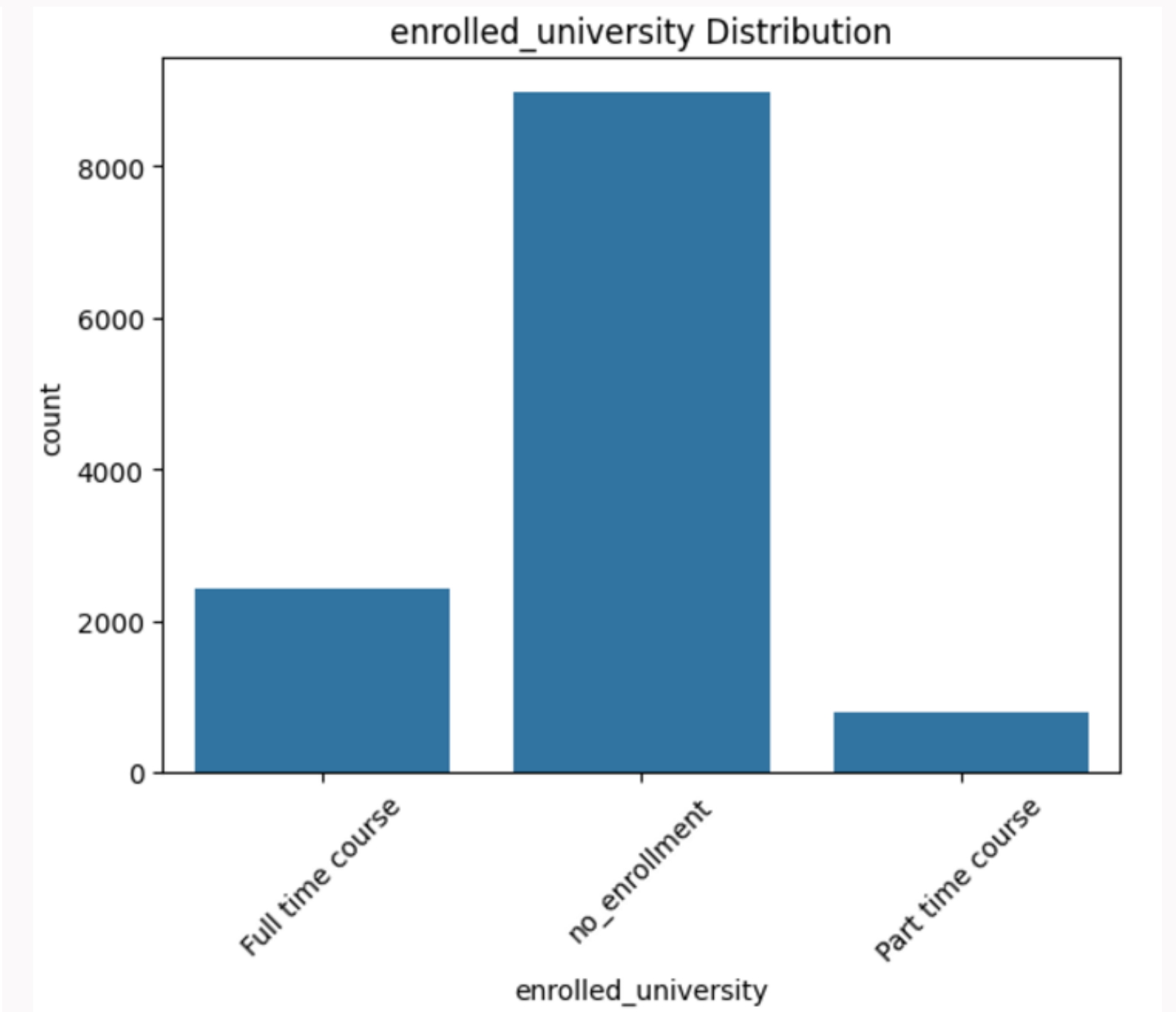
COMPANY SIZE DISTRIBUTION

주로 중소기업의 회사에 재직했던 사람들이 많으며
왼쪽으로 치우친 그래프 형태를 보임



COMPANY TYPE DISTRIBUTION

유한회사 종사가 가장 많으며, 이후에 스타트업
종사가 뒤를 따른다.



ENROLLED UNIVERSITY DISTRIBUTION

대학 등록 하지 않은 경우가 가장 많으며, full-time
등록자가 그 뒤를 따름

DATASET

이직 여부 데이터 전체 데이터에 관한 고찰

	Data Type	Null Count	결측치 비율
city_development_idx	float64	0	0%
gender	object	2917	23.43%
relevent_experience	object	0	0%
enrolled_university	object	257	2.06%
education_level	object	315	2.53%
major_discipline	object	1866	14.99%
company_size	object	3852	30.93%
last_new_job	object	273	2.19%
training_hours	int64	0	0%
city_dev_group	category	0	0%

결측치가 높은 컬럼에 대해 최빈값 또는 평균값으로 대체

데이터의 분포가 고르지 않은 컬럼은 정규화 진행

새로운 feature 추가 -> SHAP 분석 결과 높은 상관관계 추출

CONCEPT DEFINITION

다차원적인 Metrics 성능 점수 도출 방안을 마련하고
모델의 성능을 끌어올린다.

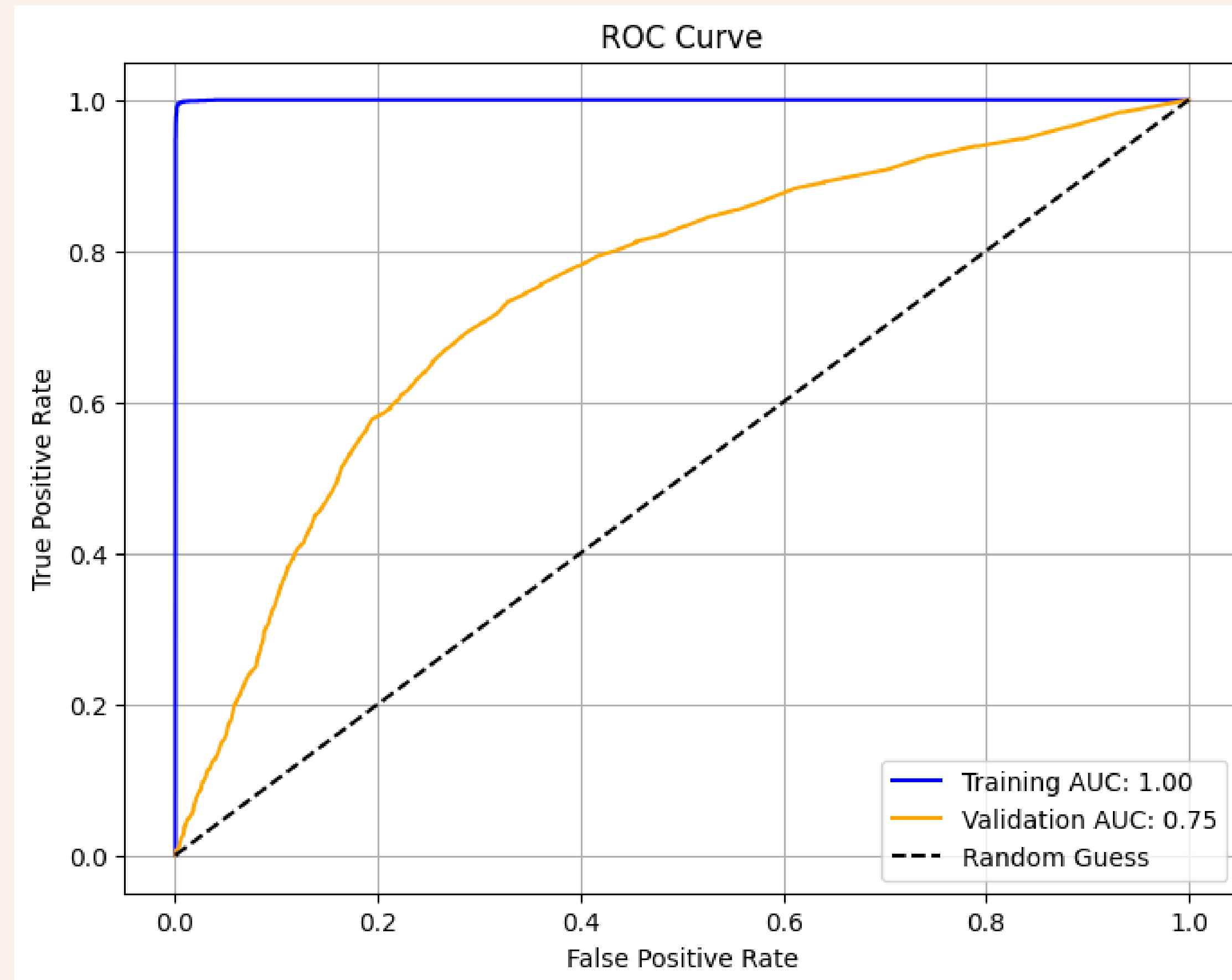
ROC Curve 비교

훈련 AUC > 0.95

검증 AUC < 0.85



목표1. 모델의 과대적합을 해결



Kappa 통계 분석

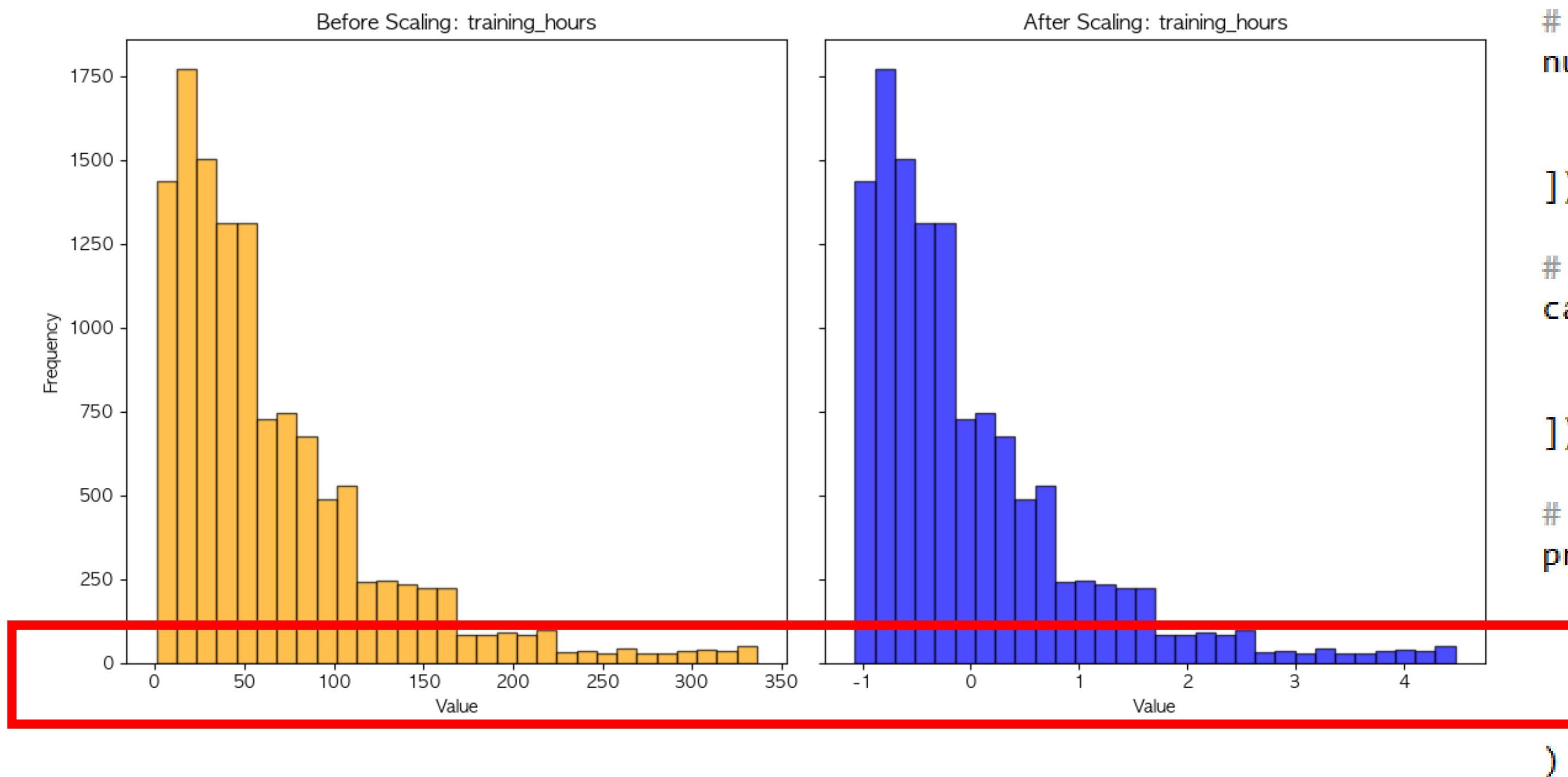
훈련 데이터 = 0.99

검증 데이터 = 0.30

목표2. 1에 가까울수록 높은
일치도 -> 모델 성능 개선

DATA PREPROCESSING

Scale이 서로 다른 Feature 어떻게 해결할 것인가?



```
# 수치형 데이터 전처리 파이프라인 정의
numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])

# 범주형 데이터 전처리 파이프라인 정의
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

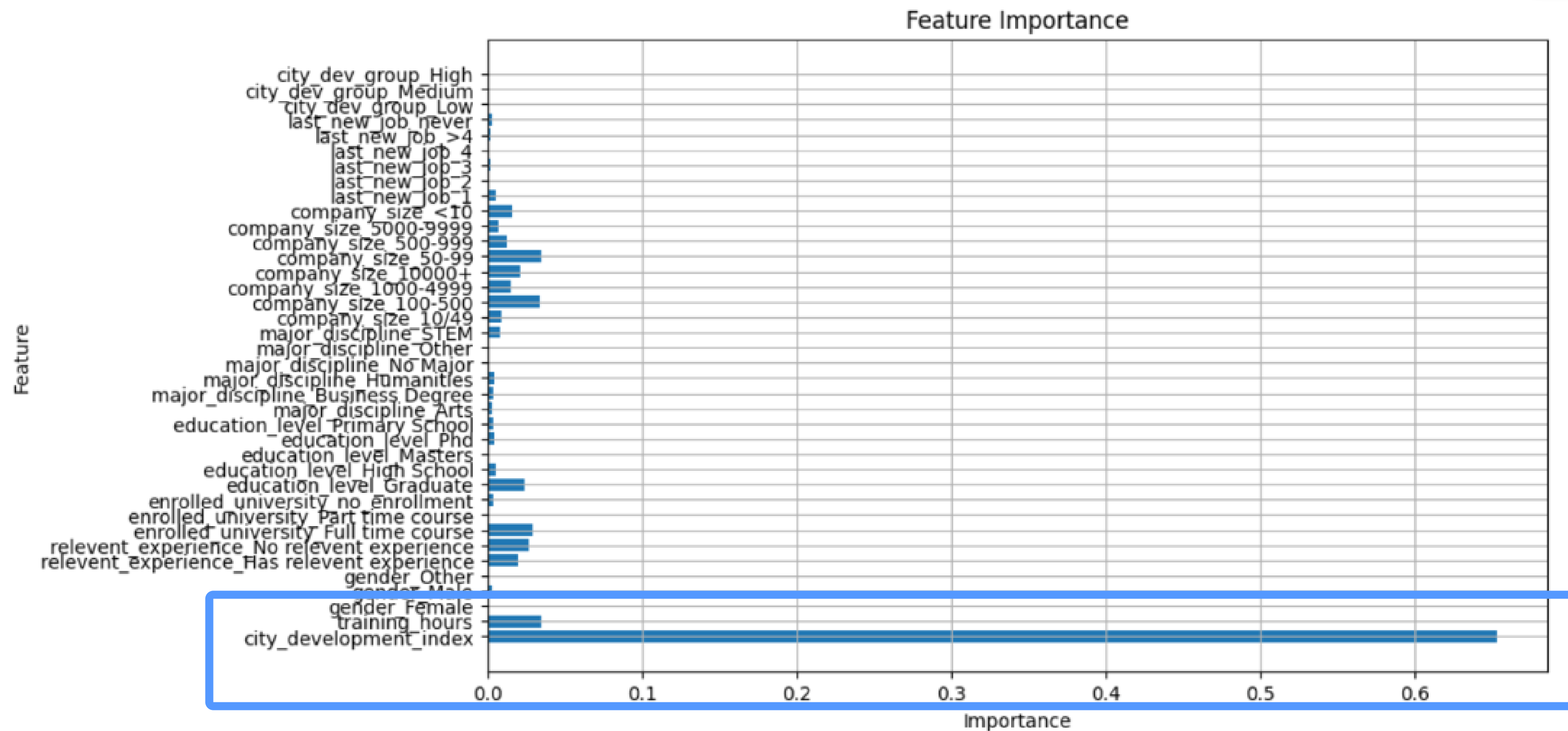
# 전체 전처리 파이프라인 구성
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ]
)
```

Scale이 서로 다른 데이터

특성의 스케일 차이를 줄이기 위해서 StandardScaler를 이용,
수치형 데이터를 평균 0, 표준편차 1로 변환합니다

DATA PREPROCESSING

부족한 Feature 어떻게 해결할 것인가?



새로운 Feature 추가

- CITY_DEV_GROUP

도시의 발전정도를 구분

0 ~ 0.7 : Low

0.7 ~ 0.85 : Medium

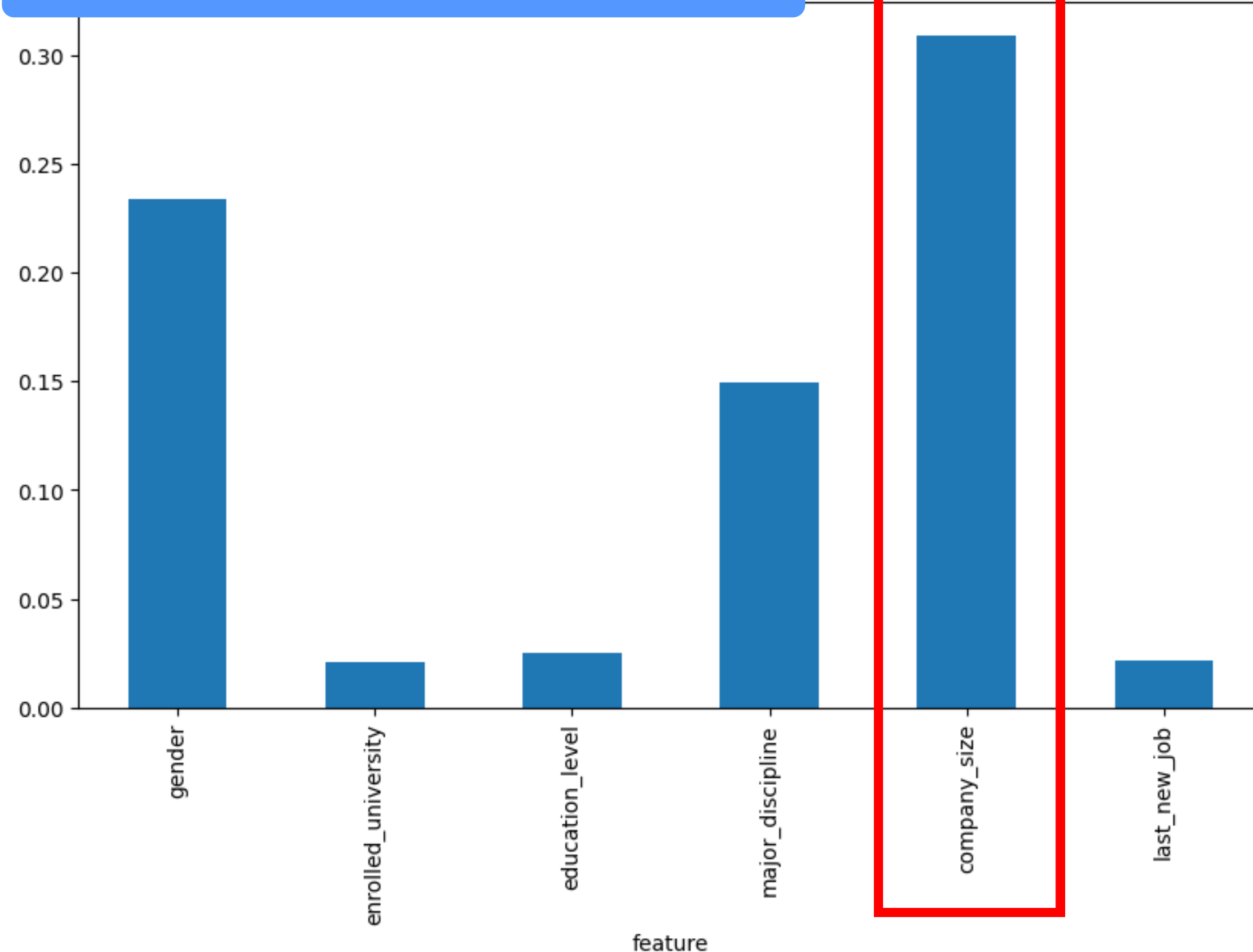
0.85 ~ 1 : High

```
x_train['city_dev_group'] = pd.cut(x_train['city_development_index'], bins=[0, 0.7, 0.85, 1], labels=['Low', 'Medium', 'High'])
x_test['city_dev_group'] = pd.cut(x_test['city_development_index'], bins=[0, 0.7, 0.85, 1], labels=['Low', 'Medium', 'High'])
```

DATA PREPROCESSING

이상치와 결측치 처리하기

각 Feature들의 결측치 비율



```
# 수치형 데이터 전처리 파이프라인 정의
numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])

# 범주형 데이터 전처리 파이프라인 정의
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

# 전체 전처리 파이프라인 구성
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ]
)
```

SimpleImputer를 사용하여 데이터의 결측치 값을 평균값 및 최빈값으로 반영

Model 01

머신러닝 모델 : 앙상블 스택킹 기법을 이용한 모델 성능 고도화

다양한 개별 모델의 예측력을 결합하여 성능을 향상시키기 위해서

```
# 투표 기반 앙상블 모델 정의 수정
def create_voting_model(models):
    estimators = []
    for name, pipeline in models.items():
        if name != 'Neural Network': #Neural Network 모델 제외
            estimators.append((name, pipeline.named_steps['classifier']))

    voting_clf = VotingClassifier(
        estimators=estimators,
        voting='soft',
        n_jobs=1 # 병렬 처리 비활성화
    )

    return Pipeline([
        ('preprocessor', preprocessor),
        ('classifier', voting_clf)
    ])

# 스택킹 모델 정의 수정
def create_stacking_model(models):
    estimators = []
    for name, pipeline in models.items():
        if name != 'Neural Network': # Neural Network 제외
            estimators.append((name, pipeline.named_steps['classifier']))

    # n_jobs를 1로 설정하여 병렬 처리 비활성화
    stacking_clf = StackingClassifier(
        estimators=estimators,
        final_estimator=LogisticRegression(),
        n_jobs=1, # 병렬 처리 비활성화
        verbose=1
    )

    return Pipeline([
        ('preprocessor', preprocessor),
        ('classifier', stacking_clf)
    ])
```

부스팅 기법

XBoostClassifier

부스팅 기법

GradientBoostClassifier

부스팅 기법

LGBMClassifier

범주형 데이터

CatBoostClassifier



Ensemble

VotingClassifier

Ensemble

StackingClassifier

Model 01

최적의 파라미터 구하는 방법 : Optuna

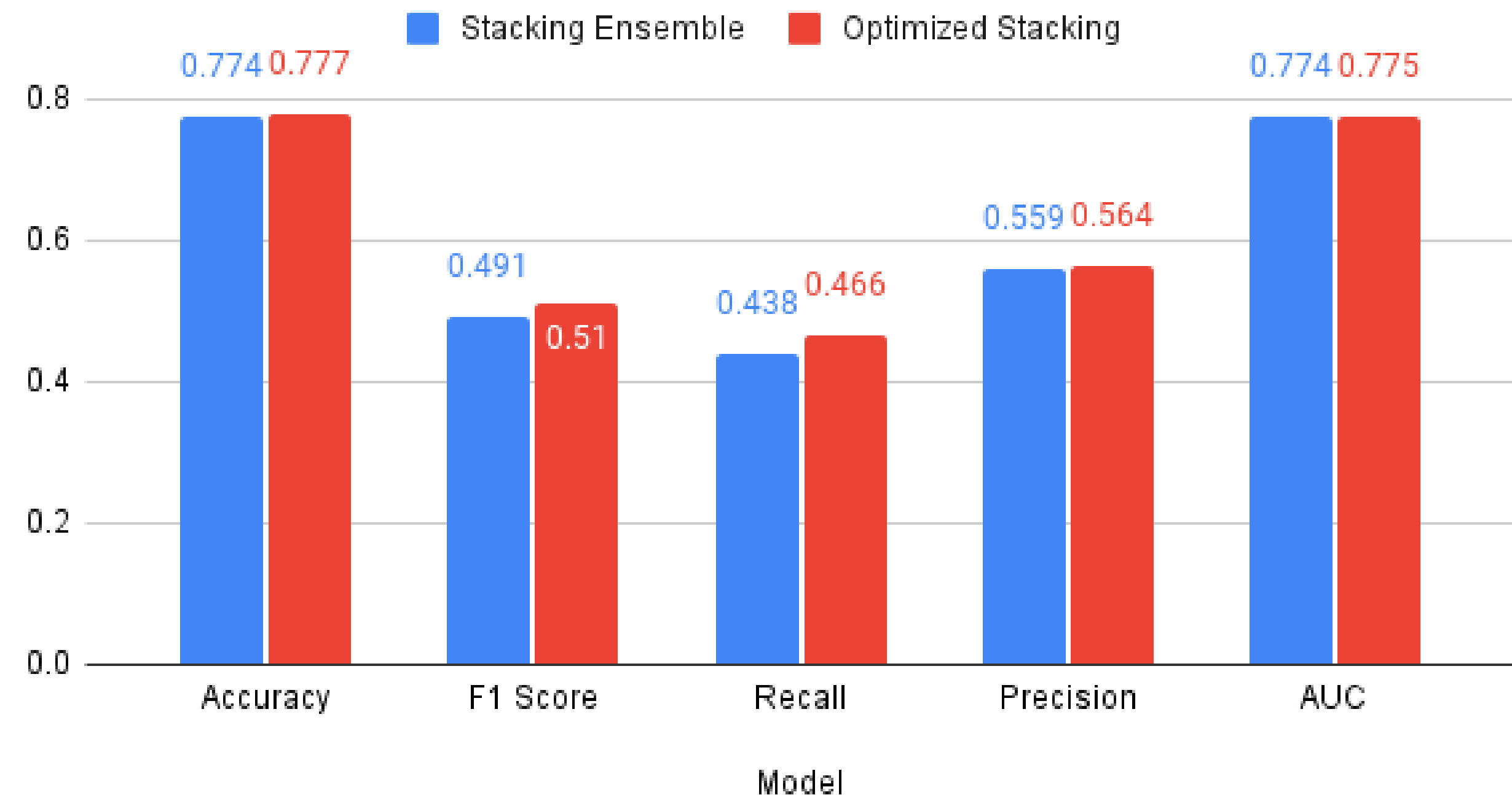
```
print("\nOptuna를 사용한 하이퍼파라미터 최적화 중...")
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=5)

best_params = study.best_params
optimized_models = {
    'XGBoost': Pipeline([
        ('preprocessor', preprocessor),
        ('classifier', XGBClassifier(
            n_estimators=best_params['xgb_n_estimators'],
            max_depth=best_params['xgb_max_depth'],
            learning_rate=best_params['xgb_learning_rate'],
            random_state=42
        ))
    ]),
    'LightGBM': Pipeline([
        ('preprocessor', preprocessor),
        ('classifier', LGBMClassifier(
            n_estimators=best_params['lgb_n_estimators'],
            max_depth=best_params['lgb_max_depth'],
            learning_rate=best_params['lgb_learning_rate'],
            random_state=42
        ))
    ]),
    'CatBoost': Pipeline([
        ('preprocessor', preprocessor),
        ('classifier', CatBoostClassifier(
            iterations=best_params['cat_iterations'],
            depth=best_params['cat_depth'],
            learning_rate=best_params['cat_learning_rate'],
            random_state=42,
            verbose=False
        ))
    ]),
    'GradientBoosting': Pipeline([
        ('preprocessor', preprocessor),
        ('classifier', GradientBoostingClassifier(
            n_estimators=best_params['gb_n_estimators'],
            max_depth=best_params['gb_max_depth'],
            learning_rate=best_params['gb_learning_rate'],
            random_state=42
        ))
    ])
}

print("\n최적화된 스택킹 모델 학습 및 평가 중...")
optimized_stacking = create_stacking_model(optimized_models)
```

optuna 라이브러리 적용 시 더 나은 파라미터 도출

Stacking Ensemble, Optimized Stacking



Model 01

머신러닝 모델 성능 평가

	accuracy	f1-score	recall	precision	auc
XGBoost	0.872572525	0.71246957	0.638081395	0.806491121	0.927880769
GradientBoosting	0.79801007	0.533886584	0.46753876	0.622179239	0.814994329
LGBM	0.833972668	0.627988182	0.566375969	0.70464135	0.891716004
CatBoost	0.839127308	0.629281768	0.551841085	0.732005141	0.884507214
Voting Ensemble	0.839606809	0.633223684	0.559593023	0.729166667	0.895581439
Stacking Ensemble	0.809877727	0.561393805	0.491763566	0.653994845	0.849854975
Optimized Stacking	0.794054184	0.534417344	0.477713178	0.606396064	0.813236468

Model 02

모델 성능 개선기: 신경망 모델을 활용한 성능 분석

```
def create_nn_model():  
    from tensorflow.keras.models import Sequential  
    from tensorflow.keras.layers import Dense, Dropout, Input  
  
    model = Sequential([  
        Input(shape=(preprocessor.get_feature_names_out().shape[0],)),  
        Dense(100, activation='relu', kernel_regularizer=l2(0.01)),  
        Dropout(0.3),  
        Dense(50, activation='relu', kernel_regularizer=l2(0.01)),  
        Dropout(0.2),  
        Dense(1, activation='sigmoid')  
    ])  
  
    model.compile(  
        optimizer='adam',  
        loss='binary_crossentropy',  
        metrics=['accuracy']  
    )  
    return model  
  
nn_pipeline = make_pipeline(  
    preprocessor,  
    KerasClassifier(  
        model=create_nn_model,  
        epochs=100,  
        batch_size=32,  
        verbose=0  
    )  
)
```

Standard Scaler 적용

스케일 차이 학습 불균형 해소,
경사하강법 안정적 수렴 도움

Dropout 레이어 추가

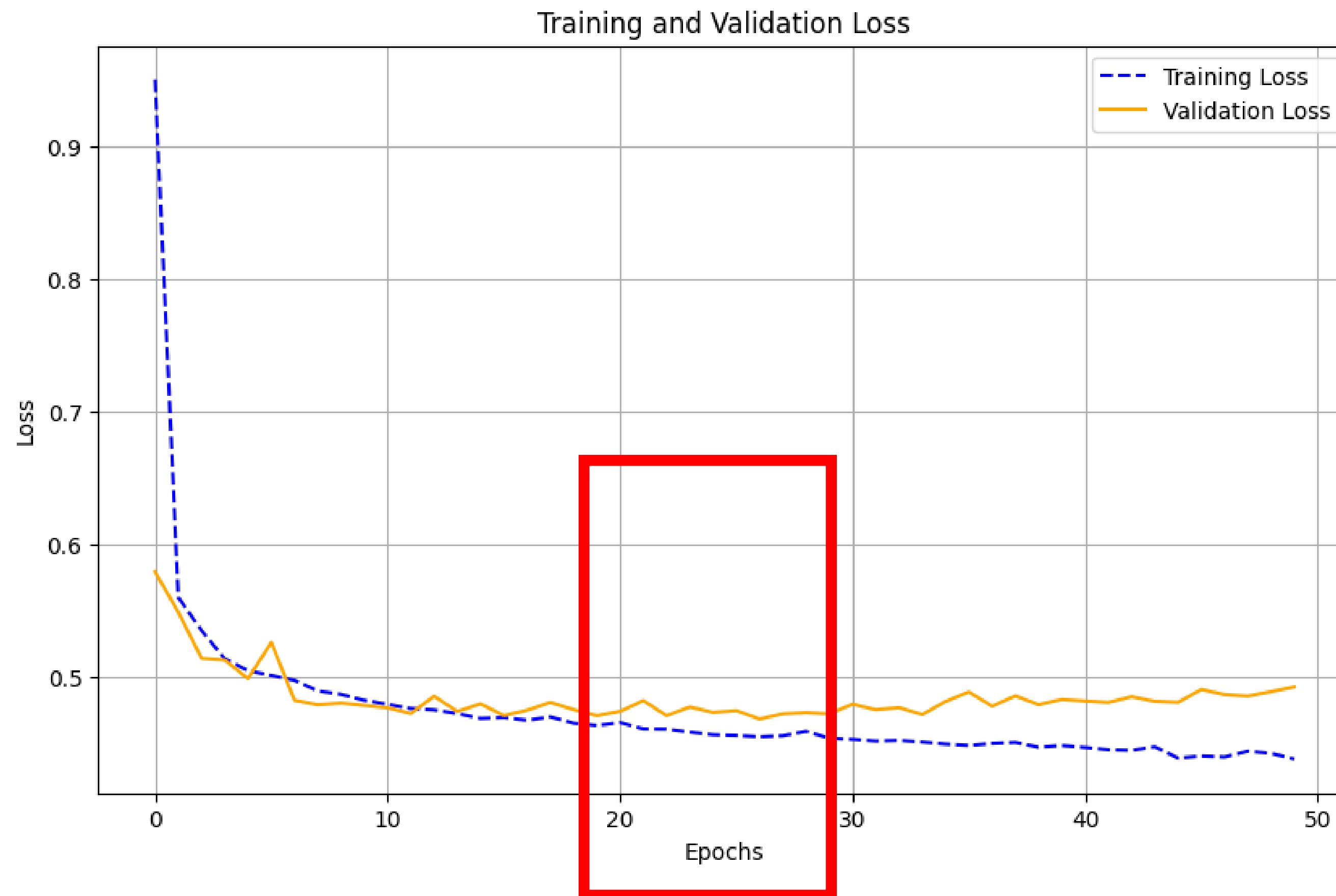
일부 뉴런 비활성화 -> 특정 뉴런에
과도하게 의존하지 않음을 유도

L2 정규화

훈련 데이터의 노이즈까지 학습 방지

Model 02

Early Stopping : 과적합을 방지하기 위한 전략



검증 손실이 개선되지 않을 때 학습 조기 중단

Epoch 20 ~ 30 지점

검증 손실 (loss) 이
일정 epoch 이후 다시 증가

모델이 훈련데이터에 과적합
되고 있을 가능성 시사

최적의 Early-Stopping 지점 도출

Model 02

Early Stopping : 성능 향상 지표

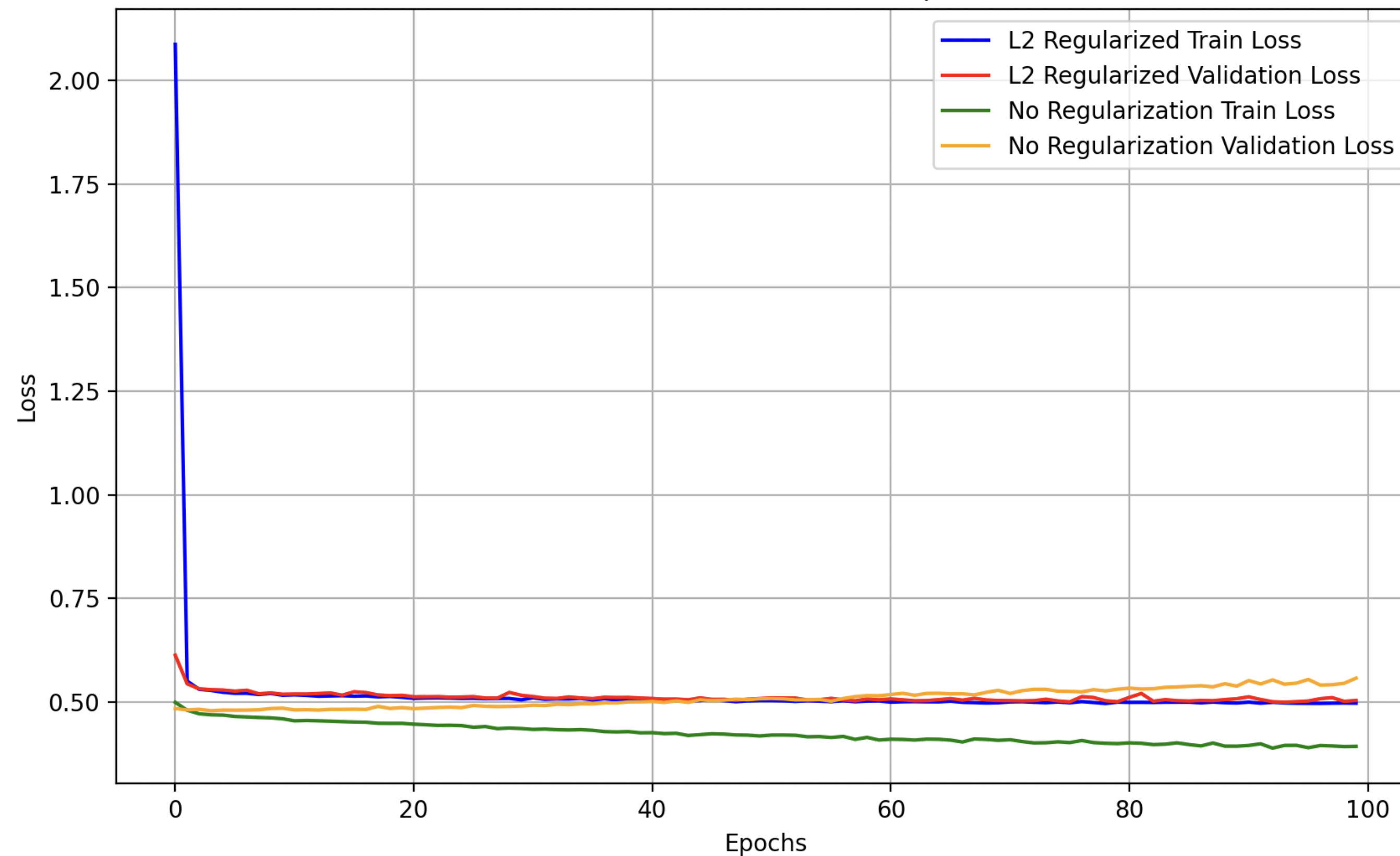
	accuracy	f1-score	recall	precision	auc
early stopping 적용 전	0.76	0.46	0.40	0.55	0.77
early stopping 적용 후	0.77	0.49	0.43	0.57	0.78

더 나은 성능
평가 점수 도출

Model 02

L2 정규화 : 일반화 성능을 높이고 과적합(overfitting)을 방지

Train and Validation Loss Comparison



데이터에 과도하게 의존적이지 않음

```
# L2 정규화 있는 모델 학습
print("L2 정규화 있는 모델 학습 중...")
model_with_l2 = create_nn_model(l2_value=0.05)
history_l2 = model_with_l2.fit(
    X_train_preprocessed,
    Y_train,
    epochs=100,
    batch_size=32,
    validation_data=(X_val_preprocessed, Y_validation),
    verbose=0
)
```

L2 - Loss의 감소가 완만하게 진행

미적용 모델 - Loss 감소가 급격하게 진행

Model VS

모델 간 성능 비교

	accuracy	f1-score	recall	precision	auc
머신러닝 모델	0.794054184	0.534417344	0.477713178	0.606396064	0.813236468
신경망 모델	0.784943659	0.510638298	0.453488372	0.584269663	0.777678983

※ Optuna 적용

※ early-stopping 적용

머신러닝 모델에서
더 좋은 성능 도출

THANK YOU

경청해주셔서 감사합니다.

지금까지 G09 김현민, 윤성빈, 최찬영이었습니다 :)